Discovery Executive Summary

Project: disocvery-7aug · **Generated:** 07/08/2026, 12:09:00

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	HIGH RISK

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), and Duplicated Domain Logic (H10); no hotspot in this category reached God-class or raw-SQL severity.

EXECUTIVE SUMMARY

The Klearcom monolith is a small, cleanly-structured codebase (no God classes, no raw SQL, no circular dependencies) whose architectural risk is concentrated in **missing middle tiers**: there is **no repository layer** (0 repository classes) and **no application-service layer** for the two product domains, so controllers and a shared `RealTimeTestService` reach directly into Eloquent models at 35+ call sites. The most damaging pattern is **duplicated domain logic** — the IVR `buildTree` recursion is copy-pasted across two controllers and the reachability-percentage formula is re-implemented in four places (`ConnectController`, `RealTimeTestService`, `LegacyReportController`, and the Node `dev-api/realtime.js`), so a single rule change must be edited in four files or silently diverge. **Bounded contexts are declared but unenforced**: `backend/app/Modules/Discovery` and `Modules/Connect` contain only `AGENTS.md`, while all real code lives in flat shared `App\Models` / `App\Http\Controllers` namespaces, and cross-domain readers (`DashboardController`, `LegacyReportController`) query both domains' tables directly with no anti-corruption layer. The frontend is healthier (Zustand + React Query, a shared transport client) but carries a class-component interval leak (`LegacyMonitorPoller`) and a boundary-less throwing

widget (`LegacyDashboardWidget`). Both layers were covered: **6 backend controllers / 4 models / 2 services** and **10 frontend components + 1 hook**.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg; 2 controllers hold KPI/business logic	MODE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 direct model-access points; 0 app services	HIGH
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 repository classes; 35+ direct Eloquent access points	HIGH
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (`LegacyDataMapper` extract-based mapper)	MODE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (no raw SQL / DB:: in controllers)	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest 165 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	3 (Dashboard, LegacyReport, RealTimeTestService); Modules/ empty	MODE
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	Cross-domain reads of 4/5 tables via reporting/dashboard, no ownership/ACL	MODE
H10	Duplicated Domain Logic *(additional)*	Duplicated logic algorithms across layers	0	1–2	>2	2 algorithms across 6 sites (`buildTree` ×2, reachability ×4)	HIGH
H11	Dual Parallel Backends *(additional)*	Parallel implementations of one API surface	0	1	>1	1 (Laravel `backend/` + Node `dev-api/`)	MODE
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	81 LOC avg; ConnectPage 222 & DiscoveryPage 176 exceed 150	MODE
F2	Missing Frontend	Components w/ inline API calls	<10	10–20	>20	6 (thin transport client exists; no per-domain service)	GOOD

	Service/Data Layer						
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest 222 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤1 level; small Zustand store (2 fields)	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (`LegacyMonitorPoller` ; boundary-less `LegacyDashboardWidget`)	MODE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Introduce `DiscoveryService` / `ConnectService` / `DashboardService` ; move model orchestration out of controllers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces + Eloquent impls; bind in `AppServiceProvider` ; remove 35+ inline ORM calls	HIGH RISK	CRITICAL
H10 Duplicated Domain Logic	Extract `IvrTreeBuilder` + `ReachabilityCalculator` ; replace `buildTree` ×2 and reachability ×4	HIGH RISK	HIGH
H1 Fat Controllers	Move `carrierSummary` / `checks` KPI math into services; drop `extract()`	MODERATE	HIGH
H8 Domain Boundary Violations	Populate empty `Modules/*` ; route cross-domain reads through published services/DTOs	MODERATE	HIGH
F5 Legacy Component Patterns	Rewrite `LegacyMonitorPoller` functional w/ cleanup; add `ErrorBoundary` ; convert widget to React Query	MODERATE	HIGH
H5 Shared Utility Abuse	Replace `LegacyDataMapper` / `extract()` with typed DTOs	MODERATE	MEDIUM
H9 Shared Database Coupling	Add per-domain summary services + read-model DTOs (ACL) for dashboard/reporting	MODERATE	MEDIUM
H11 Dual Parallel Backends	Reduce `dev-api` to a fixture/mock or converge both on one OpenAPI contract	MODERATE	MEDIUM
F1 Business Logic in Components	Move "run test + invalidate" workflow into a hook; split `ConnectPage` / `DiscoveryPage`	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable domain logic** — a `ReachabilityCalculator` and `IvrTreeBuilder` plus repository interfaces make the core rules unit-testable without MariaDB, and eliminate the 6-site duplication that currently causes silent drift.
- **Thin, single-responsibility controllers** — controllers translate HTTP ↔ application-service calls only, so the KPI/reachability rules live in one owned place instead of being copy-pasted across HTTP handlers.
- **Enforced bounded contexts** — moving code into the declared `Modules/Discovery` and `Modules/Connect` and routing cross-domain reads through published DTOs makes "extract Connect into its own service" a realistic future step and stops Discovery schema changes from breaking Connect reporting.
- **One source of truth** — collapsing the reachability/KPI logic (and optionally the Laravel/Node dual backend) onto a shared contract removes environment-to-environment divergence.
- **Resilient frontend** — a functional poller with cleanup and a top-level `ErrorBoundary` remove the interval leak and the whole-SPA blank-out risk, while consolidating on React Query gives one consistent data-fetching pattern.

Run summary: Analyzed the `FSDKC` Klearcom monolith (Laravel 12 backend + React 19 frontend + parallel Node dev-api) across both layers. Overall rating **High Risk**, driven by the absent service/repository tiers and duplicated domain logic. The complete report — including all §1.2 evidence and the five §1.3 Mermaid diagrams — is saved to `docs/discovery/01-architecture-design.md` at the workspace root; the orchestration UI will render it to `01-architecture-design.pdf`.